

System Skills A03 Mysterious Cypher

Waranyu Kittikamet (6780676)

1. Determine the Key

The key is computed in the first few lines of **main**:

- **movl num1(%rip), %eax**: This loads the value num1 (2) into the **%eax** register.
- **movl num2(%rip), %ecx**: This loads the value num2 (-5) into the **%ecx** register.
- **addl %ecx, %eax**: This adds the value in **%ecx** to **%eax**. $2 + (-5) = -3$.
- **movb %al, %bl**: This takes the lowest 8 bits of **%eax** (which is **%al**) and stores it in **%bl**. **%bl** now holds our decryption key.

Final Key Value: The computed decryption key is **-3**.

2. Recover the Plaintext

The ciphertext provided is **ddbww**. The program processes this string character by character in the **decrypt_loop** using **movzbq (%rdi), %rax** to move the current character into **%rax**, then testing to see if it hits the end of the string using **testb %al, %al**. If the end of the string has not been hit, then we proceed to the core logic of the decryption, which happens at **subb %bl, %al** and **movb %al, (%rdi)**:

subb %bl, %al

- **%al** holds the current ASCII character from the ciphertext.
- **%bl** holds our key (-3).
- The instruction subtracts **%bl** from **%al**. Since it's subtracting a negative number the operation is **ASCII value + 3**.

movb %al, (%rdi)

- **%al** holds the current ASCII character from the ciphertext after decryption.
- **(%rdi)** is the pointer to the original ASCII character from the ciphertext.
- The instruction replaces **(%rdi)** with **%al**.

After this, we move to the next character in the ciphertext using **inc %rdi** and then jump to the beginning of **decrypt_loop**.

The loop repeats for each character in **ddbww**:

- **d** (ASCII 100) + 3 = 103 → **g**
- **d** (ASCII 100) + 3 = 103 → **g**
- **b** (ASCII 98) + 3 = 101 → **e**
- **w** (ASCII 119) + 3 = 122 → **z**

Original Plaintext Message: The decrypted string is **ggez**.

3. What I Discovered:

How %al, %rax and %eax are all connected

When analyzing between **main** and **decrypt_loop** I was initially confused on why in **main**, **%al** at the end was -3, yet in **decrypt_loop** it acts as the individual letters in the encrypted message. After looking through my notes and previous slides, I eventually realized that **%al**, **%eax**, and **%rax** are all connected. **%rax** is the whole register, **%eax** is the bottom half of the register and **%al** is the bottom 8 bits of the register. By doing **movl num1(%rip), %eax** and **movzbq (%rdi), %rax** we are affecting **%al** too.

How the decrypting process works

This also explains how **%al** is used in decrypting the message. After the message, (**%rdi**), gets put into **%rax**, **%al** is also affected by this and is representing the lower 8 bits of **%rax**: the first letter in the encrypted message. By doing **inc %rdi** we essentially move where **%al** represents, thus moving from one character to the next, making decrypting each letter using **subb %bl, %al** work.